Sample code review report

Sonar demo · inventory_service.py

Codebase	Sonar demo · inventory_service.py	Generated	29 Oct 2026, 00:00 UTC
Model	nvidia/nemotron (sample)	Scope	1 files · 191 lines · 1 batch

1	191	0	0	$0.0000
Files reviewed	Lines reviewed	Input tokens	Output tokens	Est. cost (USD)

Ratings found in the report tables

HIGH 2 | MEDIUM 3 | LOW 4 | INFO 1

SAMPLE REPORT. This file shows the layout of a QuCodRev PDF report for the QuSonar demo program (Sonar/inventory_service.py). It was prepared by hand for the guided tour; no model was called, so tokens and cost are 0. Run QuCodRev on your own code to get a real report.

Code review: inventory_service.py

Summary

The core `Inventory` class is small, readable and well guarded: every stock movement goes through one validation helper, shipping refuses to go negative, and the movement history makes stock levels auditable. The defects sit in the helper functions at the end of the file, where four are real bugs (an exception that is never raised, a comparison that is always false, a value that is silently overwritten, a dead default) and the rest are maintainability problems.

Findings

#	Rating	Location	Finding	Fix
1	**High**	`validate_sku`, line 173	`InventoryError("A SKU is required")` is created but never raised, so an empty SKU is reported as valid.	`raise InventoryError("A SKU is required")`
2	**High**	`over_capacity`, line 178	`used_slots - used_slots` is always `0`, so the function returns `False` for any input.	Compare `used_slots > capacity`.
3	**Medium**	`warehouse_routes`, lines 185–186	`routes["active"]` is assigned twice; the primary route is lost and only the fallback is kept.	Store them under separate keys (`active`, `fallback`).
4	**Medium**	`sync_timeout`, lines 133–134	The first assignment (`30`) is dead; the effective default is `60`. `int()` also raises on a bad value.	Keep one default and validate the option.
5	**Medium**	`restock_candidates`, lines 162–167	Five nested `if` statements hide a single condition and make it hard to test.	Combine into one guard expression or a predicate function.
6	**Low**	`build_stock_query`, line 104	Fourteen positional parameters; callers can easily swap arguments of the same type.	Use keyword-only arguments or a dataclass.
7	**Low**	`shipping_plan`, lines 139–146	The `express` and `priority` branches are identical.	Merge them (`if mode in ("express", "priority")`).
8	**Low**	`count_movements`, line 128	`debug_snapshot` is computed and never used (and costs a full `repr`).	Remove the variable.
9	**Low**	`format_sku_label`, line 123	Parameter `legacy_mode` is unused.	Remove it or implement the legacy format.
10	**Info**	`stock_error_text`, lines 152–156	Every branch returns the same text, so callers cannot tell the cases apart.	Return a distinct message per kind.

Detail: finding 1

```
def validate_sku(sku):
    if not sku:
        InventoryError("A SKU is required")   # created, never raised
    return {"sku": sku, "valid": True}
```

Impact. Any caller that relies on `validate_sku` accepts empty SKUs; they then fail later, far from the cause, or are written to the history as a product with no identifier.

Fix.

```
def validate_sku(sku):
    if not sku:
        raise InventoryError("A SKU is required")
    return {"sku": sku, "valid": True}
```

What is working well

Area	Why it is good
Single validation path	`receive` and `ship` both call `_check`, so unknown SKUs and non-positive quantities are rejected the same way.
No negative stock	`ship` checks availability before changing state, so a failed shipment leaves stock and history untouched.
Audit trail	Every change appends a `Movement` with a UTC timestamp; `movements_for` can rebuild any product's history.
Clear errors	`InventoryError` messages name the SKU and the quantities involved.

Next steps

1. Fix findings 1–3 first: they change behaviour.
2. Add unit tests for `validate_sku`, `over_capacity` and `warehouse_routes` that fail before the fix.
3. Tidy the remaining helpers (findings 4–10) in one refactoring change.

Files reviewed

File	Lines
inventory_service.py	191

About this report

Sample prepared for the QuCoRe guided tour. A real report lists the model, tokens and cost of the run; its claims should still be checked against the code.

Sample report: no model call was made.